

专题：从Web应用看GUI与性能测试
——结合Selenium、Jmeter、大模型等工具



Web软件（网站）应用测试
背景与问题

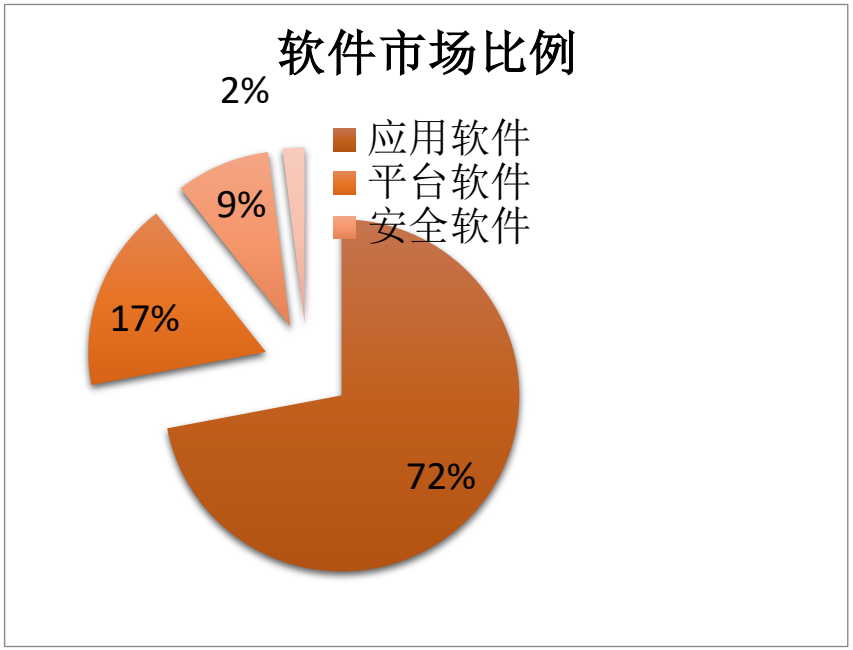
Web软件（网站）测试

1. Web软件的特点
2. Web网站的特性
3. 网页的测试
4. 网站的测试

1 Web软件的特点

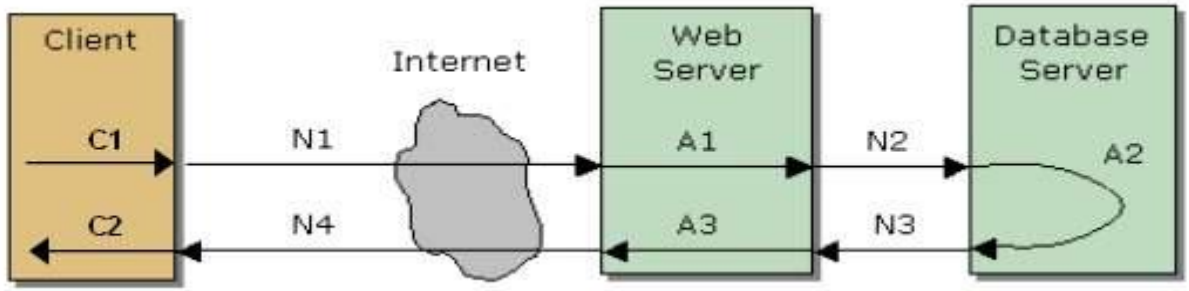
软件的分类

- 平台软件、中间件、应用软件
- 而应用软件中，Web软件占据了主要份额



Web软件的特点?

- 网络化
- 多用户





2 Web网站的特点

2.1 Web网站的概念

2.2 Web网站的特点

2 Web网站的特性

2.1. Web网站的概念

- 大多数Web网站都是采用B-S结构 。
- 能够交付一组复杂的内容和功能给大量的终端用户。
- Web网站测试是用于测试高质量Web应用系统的过程，它借用了许多传统软件测试和系统测试的概念和原理。



2.2 Web网站的特点

- 网络集约性
- 内容驱动性
- 持续演化性
- 即时性
- 安全性
- 美观性

[illegible]

[illegible]

[illegible]

3 网页测试

网页测试包括以下内容：

3.1. 功能测试

3.2. 可用性测试

3.1 功能测试

功能测试包括以下内容：

- 链接测试
- 表单测试
- 数据校验
- Cookies 测试

什么是链接?

链接是Web 网站的一个主要特征，它是在页面之间切换和引导用户去一些未知地址页面的主要手段。

什么是Cookies?

Cookie是一个由网页服务器放在您硬盘上的非常小的文本文件. 它本质上就像您的身份证明一样，并且不能像代码那样被执行或被用来散布病毒。它只能被您使用并且只能由提供的服务器读取。

Cookies 测试

使用Cookie的目的:

- 帮您节约时间。如果您自定义页面，或注册产品或服务。cookie记住您的身份,当下一次您再次访问的时候，将显示您需要的信息，将帮您填入任何您已经回答过的问题。

- Cookies 通常用来存储用户信息和用户在某些应用系统上的操作序列，当一个用户使用Cookies访问了某一个应用系统时，Web 服务器将发送关于用户的信息，并把该信息以Cookies 的形式存储在客户端计算机上，这可用来创建动态和自定义页面或者存储登录等信息。

Cookies 测试

测试内容：

- Cookies是否能正常工作；
- Cookies是否按预定的时间进行保存；
- 刷新对Cookies 有什么影响等。

举例：

- 如果在cookies 中保存了注册信息，应确认该cookie 能够正常工作而且已对这些信息进行加密。
- 如果使用cookie 来统计次数，需要验证次数累计正确。

问题： Cookies 造成的负面作用是什么?

3.2. 可用性测试

可用性测试包括：

① 导航测试：导航描述了用户在一个页面内操作的方式

② 图形测试：一个Web 网站的图形可以包括图片、动画、边框、颜色、字体、背景、按钮等。

③ 内容测试：内容测试用来检验Web 网站提供信息的正确性、准确性和相关性。

④ 整体界面测试：对整个Web 系统的页面结构设计的测试，是用户对系统的一个整体感受。

① 导航测试的内容

- 导航是否直观?
- Web 系统的主要部分是否可以通过主页访问?
- Web系统是否需要站点地图、搜索引擎或其他的导航器帮助?
- 测试Web 系统的页面结构;
- 导航条、菜单、连接的风格是否一致?
- 各种提示是否准确，确保用户凭直觉就知道是否还有内容，内容在什么地方。
- 最好让最终用户参与导航测试，效果将更加明显。

② 图形测试

- 要确保图形有明确的用途，图片或动画不要胡乱地堆在一起，以免浪费传输时间。图片尺寸要尽量地小，并且要能清楚地说明某件事情。
- 验证所有页面字体的风格是否一致。
- 景颜色应该与字体颜色和前景颜色相搭配。
- 图片的大小和质量也是一个很重要的因素，一般采用JPG 或GIF 压缩。

4 网站测试

网站测试在网页测试的基础之上还包括：

4.1 功能测试；

4.2 性能测试；

4.3 安全性测试；

4.4 兼容性测试

4.1 功能测试

功能测试包括以下内容：

1） **数据库测试**

在Web 应用中，最常用的数据库类型是关系型数据库，可以使用SQL 对信息进行处理。

2） **WEB网站特定的功能需求测试**

依据需求规格说明书对WEB网站特定的功能需求进行验证。

3） **设计语言测试**

不同的Web 设计语言版本的差异可以引起客户端或服务器端严重的问题；

例如使用Java、JavaScript、ActiveX、VBScript或Perl等开发的应用程序也要在不同的版本上进行验证。

整体界面测试

例如，当用户浏览Web 网站时，应考虑

- 是否感到舒适?
- 是否凭直觉就知道要找的信息在什么地方?
- 整个Web 应用系统的设计风格是否一致?

数据库测试

两种主要数据库错误：

- 数据一致性错误

  主要是由于用户提交的表单信息不正确而造成的。
- 输出错误

  主要是由于网络传输速度或程序设计问题等引起的。

数据库测试就要针对这两种情况，分别进行测试。

思考：Web应用中的数据库测试

数据库测试是一个专题，一般包括

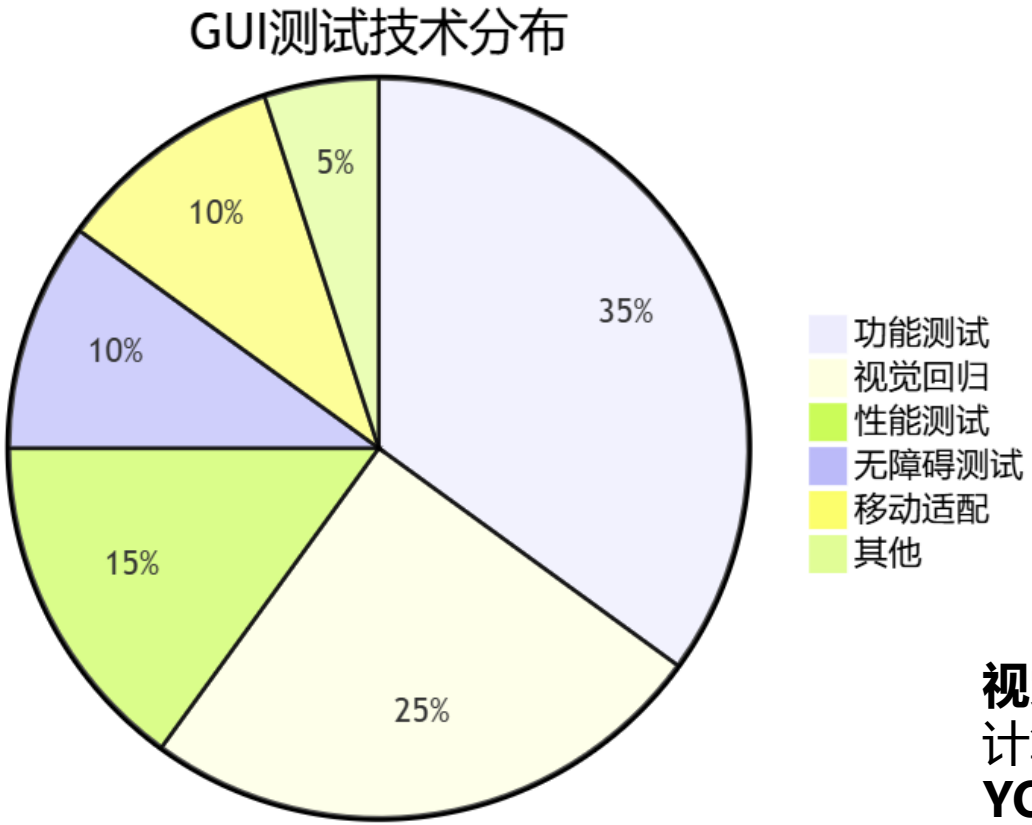
- 数据一致性
- 备份与恢复
- 数据库响应测试（查询时间）
- 数据库压力测试（数据量）
- 并发测试
- 权限管理
- ……

GUI测试

GUI测试（Graphical User Interface Testing）是一种软件测试方法，专注于验证图形用户界面（GUI）的功能、布局、视觉表现和用户体验是否符合设计需求和用户预期。它通过模拟用户与界面元素的交互（如点击按钮、输入文本、导航菜单等），检查以下方面：

- **功能正确性：**界面元素的行为是否符合预期（如按钮点击触发正确操作）
- **视觉一致性：**布局、颜色、字体等视觉元素是否与设计规范一致
- **兼容性：**界面在不同设备、操作系统、浏览器或分辨率下的表现是否正常
- **用户体验：**交互流程是否流畅，响应时间是否符合标准

GUI测试



视觉回归测试增强

计算机视觉进行视觉回归测试

YOLOv8：检测界面元素位置

OpenCV：像素级差异比对

Diff算法：结构相似性(SSIM)分析

GUI测试主要技术工具

- **提供基础的GUI测试功能测试（Functional Testing）**
- **验证界面元素的行为是否符合需求规范，包括按钮点击、表单提交、导航跳转等**

- **Selenium**

  开源、支持多种编程语言（Java/Python/C#等）、跨浏览器测试、与CI/CD集成

  适用场景：Web应用的自动化功能测试

- **Cypress**

  实时重载、自动等待机制、时间旅行调试、支持端到端测试。

  适用场景：现代Web应用的快速测试开发

- **TestComplete**

  支持桌面/Web/移动端应用、录制回放功能、对象识别引擎。

  适用场景：企业级跨平台GUI测试

GUI测试主要技术工具

- **移动端GUI测试（Mobile GUI Testing）**

- **Appium**

  跨平台（iOS/Android）、支持原生/混合应用、与Selenium兼容。

  适用场景：移动应用的自动化功能测试

- **Espresso (Android) / XCUITest (iOS)**

  官方原生框架、与IDE深度集成、执行速度快

  适用场景：移动端精准的UI交互测试

- **Detox**

  专注于React Native应用、支持异步操作同

Web应用中的GUI测试

Web应用需不需要GUI测试?

- 用户体验的第一道防线（70%用户因UI问题流失）
- 保障功能与视觉的一致性（降低30%客户投诉）
- 跨设备兼容性守护者（覆盖2000+设备/浏览器组合）

Web应用中的GUI测试

定义:Web GUI测试是针对网页或Web应用程序的图形用户界面测试

- **跨浏览器兼容性：**检查在不同浏览器（Chrome、Firefox、Safari等）和版本中的渲染一致性
- **响应式布局：**确保界面在不同屏幕尺寸（桌面、平板、手机）和分辨率下自适应显示
- **动态元素行为：**验证JavaScript驱动的交互（如表单验证、异步加载、动态内容更新）
- **前端性能：**评估页面加载速度、资源消耗（如内存/CPU占用）和渲染效率
- **安全性：**检查跨站脚本（XSS）等前端漏洞的防护措施

Web应用中的GUI测试

传统挑战

- 浏览器兼容
- 响应式布局
- 基础功能验证
- 手工测试为主

现代复杂性

- 渐进式Web应用(PWA)
- 微前端架构
- Web3D/WebGL应用
- 低代码平台动态生成



GUI测试工具介绍
——Selenium



第一部分

1. Selenium 简介
2. 什么情况下选用WebDriver
3. Webdriver对浏览器的简单操作
4. Webdirver对浏览器的支持
5. 基本操作
6. 如何对页面元素进行操作

Selenium 简介

- Selenium 是ThroughtWorks 公司一个强大的开源 Web 功能测试工具系列，
- Selenium 可以使用录制工具录制脚本，测试页面。
- Selenium 可以生成类html 代码，java 代码，ruby 代码等。
- Selenium 录制工具根据id 属性定位html 元素
- Selenium IDE 仅支持Selenium 语言。
- Selenium RC 支持很多语言，如：C#，Java，Python，Ruby 等。

Selenium 2.0

- Selenium2.0 = Selenium1.0+ webdriver
- WebDriver项目是由Simon Stewart 提出的，它是一个轻便简洁的自动化测试框架。
- WebDriver通过尝试不同的方法去解决Selenium1.0所面临的问题。不单单是使用javascript，WebDriver会使用任何一种更合适的机制来操作浏览器
- 通过更灵活的机制去操控浏览器，就能很好的绕过浏览器javascript的安全限制。当这些技术还不够用时，可以调用系统设备操作，尤其是当你需要一些键盘和鼠标操作时，通过这些技术，可以更好的模拟用户的真实浏览器操作

什么情况下选用WebDriver？

- Selenium-1.0不支持的浏览器功能
- multiple frames, multiple browser windows, popups, and alerts
- 页面导航
- 下拉
- 基于AJAX的UI元素

安装selenium说明

- Selenium无需安装环境；
- 只需在工程文件中，增加库文件：selenium-server-standalone-X.X.X，即可使用；
- 或者使用maven导入：

```
<dependencies>
    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>3.11.0</version>
    </dependency>
</dependencies>
```

- 需要对应browser和browser driver支持

Webdriver对浏览器的简单操作

A）打开一个测试浏览器

```
public class OpenBrowsers {

  public static void main(String[] args) {
    //打开默认路径的firefox
    WebDriver diver = new FirefoxDriver();

    //打开指定路径的firefox,方法1
    System.setProperty("webdriver.firefox.bin","C:\\Program Files\\Mozilla Firefox\\firefox.exe");
    WebDriver driver1 = new FirefoxDriver();

    //打开指定路径的firefox,方法2
    File pathToFirefoxBinary = new File("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
    FirefoxBinary firefoxbin = new FirefoxBinary(pathToFirefoxBinary);
    WebDriver driver2 = new FirefoxDriver(firefoxbin,null);

    //打开ie
    System.setProperty("webdriver.ie.driver","E:\\2013上半年\\Web应用项目测试\\software\\IEDriverServer_Win32\\IEDriverServer.exe");
    WebDriver ie_driver = new InternetExplorerDriver();

    //打开chrome
    System.setProperty("webdriver.chrome.driver", "E:\\2013上半年\\Web应用项目测试\\software\\chromedriver_win\\chromedriver.exe");
    System.setProperty("webdriver.chrome.bin", "C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe");
    WebDriver chrome_driver = new ChromeDriver();

    String url = "http://www.51.com";

    driver1.get(url);
    driver2.get(url);
    ie_driver.get(url);
    chrome_driver.get(url);

  }
```

Webdriver对浏览器的简单操作

B）打开1个具体的url

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public class OpenUrl {
public static void main(String[] args) {
String url = "http://www.51.com";
WebDriver driver = new FirefoxDriver();

// 用get方法
driver.get(url);

// 用navigate方法，然后再调用to方法
driver.navigate().to(url);
}
}
```

Webdriver对浏览器的简单操作

C）关闭浏览器

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public class CloseBrowser {
public static void main(String []args){
    String url = "http://www.51.com";
WebDriver driver = new FirefoxDriver();
driver.get(url);

//用quit方法
driver.quit();
//用close方法
driver.close();
}
}
```

Webdriver对浏览器的简单操作

D）返回当前页面的url和title

```
public class GetUrlAndTitle {
public static void main(String []args){

String url = "http://www.51.com";
WebDriver driver = new FirefoxDriver();

driver.get(url);

    //得到title
String title = driver.getTitle();

    //得到当前页面url
String currentUrl = driver.getCurrentUrl();

    //输出title和currenturl
System.out.println(title+"\n"+currentUrl);
}
    }
```

Webdriver对浏览器的简单操作

E）其他方法

- getWindowHandle()　返回当前的浏览器的窗口句柄
- getWindowHandles()　返回当前的浏览器的所有窗口句柄
- getPageSource()　返回当前页面的源码

Webdirver对浏览器的支持

HtmlUnit Driver

- 优点：HtmlUnit Driver不会实际打开浏览器，运行速度很快。对于用FireFox 等浏览器来做测试的自动化测试用例，运行速度通常很慢，HtmlUnit Driver 无疑是可以很好地解决这个问题。
- 缺点：它对JavaScript的支持不够好，当页面上有复杂JavaScript时，经常会捕获不到页面元素。
- 使用：

```
WebDriver driver = new HtmlUnitDriver();
```

- 在selenium3中取消了对HtmlUnit Driver的支持

Webdirver对浏览器的支持

FireFox Driver

- 优点：FireFox Dirver对页面的自动化测试支持得比较好，很直观地模拟页面的操作，对JavaScript的支持也非常完善，基本上页面上做的所有操作FireFox Driver都可以模拟。
- 缺点：启动很慢，运行也比较慢，不过，启动之后Webdriver的操作速度虽然不快但还是可以接受的，建议不要频繁启停FireFox Driver。
- 使用：

```
WebDriver driver = new FirefoxDriver();
```

Webdirver对浏览器的支持

InternetExplorer Driver

- 优点：直观地模拟用户的实际操作，对JavaScript提供完善的支持。
- 缺点：是所有浏览器中运行速度最慢的，并且只能在Windows下运行，对CSS以及XPATH的支持也不够好。
- 使用：

```
WebDriver driver = new InternetExplorerDriver();
```



基本操作

如何找到页面元素

- Webdriver的findElement方法可以用来找到页面的某个元素，最常用的方法是用id和name查找。下面介绍几种比较常用的方法。

基本操作

By ID

假设页面写成这样：

```
<input type="text" name="passwd" id="passwd-id"/>
```

那么可以照样找到页面的元素：通过id查找

```
WebElement element = driver.findElement(By.id("passwd-id"));
```

基本操作

例：

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;  import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class ByUserId {

/**
* @param args
*/
public static void main(String[] args) {
// TODO Auto-generated method stub  WebDriver dr = new FirefoxDriver();
dr.get("http://www.51.com");

WebElement element = dr.findElement(By.id("passport_51_user"));
System.out.println(element.getAttribute("title"));
}

}
```

基本操作

By Name

通过name查找：

```
WebElement element =
driver.findElement(By.name("passwd"));
```

例：

```
public class ByClassName {
public static void main(String[] args) {  WebDriver driver = new
FirefoxDriver();
driver.get("http://www.51.com");

WebElement e=dr.findElement(By.name("passport_51_user"));
System.out.println(element.getTagName());
}
}
```

基本操作

By Class Name

假设页面写成这样：

```
<div class="cheese"><span>Cheddar</span></div>
<div class="cheese"><span>Gouda</span></div>
```

可以通过这样查找页面元素：

```
List<WebElement>cheeses =
driver.findElements(By.className("cheese"));
```

基本操作

例：

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class ByClassName {
public static void main(String[] args) {
        WebDriver driver = new FirefoxDriver();
driver.get("http://www.51.com");
WebElement element = driver.findElement(By.className("username"));
System.out.println(element.getTagName());
}
}
```

基本操作

By XPATH

通过XPATH查找：

```
WebElement element
=driver.findElement(By.xpath("//input[@id='passwd-id']"));
```

基本操作

By CSS

假设CSS写成这样：

```
<div id=“food” >
<span class=“dairy” >milk</span>
<span class=“dairy aged” >cheese</span>
</div>
```

通过CSS selectors查找：

```
WebElement element =driver.findElement
(By.cssSelector( “#food span.dairy.aged"));
```

基本操作

By Link Text

假设页面元素写成这样：

```
<a
href="http://www.google.com/search?q=cheese">cheese</a>>
```

那么可以通过这样查找：

```
WebElement cheese=driver.findElement(By.linkText("cheese"));
```

如何对页面元素进行操作

输入框（text field or textarea）

找到输入框元素：

```
WebElement element =  driver.findElement(By.id("passwd-id"));
```

在输入框中输入内容：

```
element.sendKeys(“test”);
```

将输入框清空：

```
element.clear();
```

获取输入框的文本内容：

```
element.getAttribute("value")
```

如何对页面元素进行操作

单选项(Radio Button)

找到单选框元素：

```
WebElement radio=driver.findElement(By.id("BookMode"));
```

选择某个单选项：

```
radio.click();
```

清空某个单选项：

```
radio.clear();
```

判断某个单选项是否已经被选择：

```
radio.isSelected();
```



如何对页面元素进行操作

多选项(checkbox)

多选项的操作和单选的差不多：

```
WebElement checkbox =
driver.findElement(By.id("myCheckbox."));
checkbox.click();
checkbox.clear();
checkbox.isSelected();
checkbox.isEnabled();
```

如何对页面元素进行操作

按钮(button)

找到按钮元素:

```
WebElement saveButton =  driver.findElement(By.id("save"));
```

点击按钮:

```
saveButton.click();
```

判断按钮是否enable:

```
saveButton.isEnabled ();
```

如何对页面元素进行操作

左右选择框（穿梭框）

也就是左边是可供选择项，选择后移动到右边的框中，反之亦然。例如：

```
Select lang = new  Select(driver.findElement(By.id("languages")));
lang.selectByVisibleText(“English”);
WebElement addLanguage=driver.findElement(By.id("addButton"));
addLanguage.click();
```



如何对页面元素进行操作

弹出对话框(Popup dialogs)

```
Alert alert = driver.switchTo().alert();
alert.accept();
alert.dismiss();
alert.getText();
```

如何对页面元素进行操作

表单(Form)

Form中的元素的操作和其它的元素操作一样，对元素操作完成后对表单的提交可以：

```
WebElement approve =  driver.findElement(By.id("approve"));
approve.click();
```

或

```
approve.submit();//只适合于表单的提交
```

如何对页面元素进行操作

上传文件 (Upload File)

上传文件的元素操作：

```
WebElement adFileUpload =  driver.findElement(By.id("WAP-upload"));
String filePath =  "C:\test\\uploadfile\\media_ads\\test.jpg";
addFileUpload.sendKeys(filePath);
```



第二部分

1. 多个对象的定位方法
2. 层级定位
3. frame的处理
4. 弹出窗口
5. 处理select下拉框
6. 操作Cookies
7. 拖放元素

多个对象的定位方法

findElements()方法可以返回一个符合条件的元素 List组

```
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class FindElementsStudy {

    public static void main(String[] args) {
        WebDriver  driver = new FirefoxDriver();
        driver.get("http://www.51.com");

        List<WebElement> element = driver.findElements(By.tagName("input"));
        for (WebElement e : element){
            System.out.println(e.getAttribute("id"));
        }

        driver.quit();
    }
}
```

层级定位

- 层级定位的思想是先定位父元素，然后再从父元素中精确定位出其我们需要选取的子元素。

- 层级定位一般的应用场景是无法直接定位到需要选取的元素，但是其父元素比较容易定位，通过定位父元素再遍历其子元素选择需要的目标元素，或者需要定位某个元素下所有的子元素。

层级定位

```
import java.io.File;
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxBinary;
import org.openqa.selenium.firefox.FirefoxDriver;

public class LayerLocator {

   public static void main(String[] args) {

      WebDriver  driver = new FirefoxDriver();
      driver.get("http://www.51.com");


      WebElement element = driver.findElement(By.className("login"));
       List<WebElement> el = element.findElements(By.tagName("label"));
       for(WebElement e : el)
      System.out.println(e.getText());

   }

}
```



iframe的处理

selenium webdriver中提供了进入一个iframe的方法：

WebDriver org.openqa.selenium.WebDriver.TargetLocator.frame(String nameOrId);

提供了一个返回default content的方法：

WebDriver org.openqa.selenium.WebDriver.TargetLocator.defaultContent()

实例： main.html

```
<html>
<head>
<title>FrameTest</title>
</head>
<body style="background-color: #990000;">
<div id = "id1">this is a div!</div>
<iframe id = "frame"  frameborder="0" scrolling="no" style="left:0;position:absolute;" src = "frame.html"></iframe>
</body>
</html>
```

实例：frame.html

```
<html>
<head>
<title>this is a frame!</title>
</head>
            <body style="background-color: #009900;">
                <div id = "div1">this is a div,  too!</div>
<label>input:</label>
<input id = "input1"></input>
</body>
</html>
```



测试

```
public class FameTest {
public static void main(String[] args) {
        WebDriver dr = new FirefoxDriver();
        String url = "file://C:/Users/LRQ/workspace/2013-selenium-webdriver-2/main.html"
        dr.get(url);
        //在default content定位id="id1"的div
        dr.findElement(By.id("id1"));
        //此时，没有进入到id="frame"的frame中时，以下两句会报错
        dr.findElement(By.id("div1"));//报错
        dr.findElement(By.id("input1"));//报错
        //进入id="frame"的frame中，定位id="div1"的div和id="input1"的输入框。
        dr.switchTo().frame("frame");
        dr.findElement(By.id("div1"));
        dr.find
        //此时
        会报错
        dr.findElement(By.id("id1"));//报错
        //跳出frame,进入default content;重新定位id="id1"的div
        dr.switchTo().defaultContent();
        dr.findElement(By.id("id1"));
}
```

switch_to方法会new1个TargetLocator对象，使用该对象的 frame方法可以将当前识别的”主体”移动到需要定位的frame上

弹出窗口

在selenium 1.X里面得到弹出窗口是一件比较麻烦的事，特别是新开窗口没有id、name的时候。在 selenium webdriver中得到新开窗口相对简单的多，它无关新开窗口的id、name等属性。以下面的popup.html为例：

```
<span style="white-space: normal; background-color: rgb(255,
    255,  255);">test.html</span>


<html>
  <head><title>Test Popup Window</title></head>

  <body>
    <a id = "51" href = "http://www.51.com/" target = "_blank">Let's
    go!</a>
  </body>

</html>
```

测试

```
public class PopupWindowTest {
public static void main(String[] args) {  WebDriver dr = new FirefoxDriver();
String url ="file://C:/Users/LRQ/workspace/2013-selenium-webdriver-2/popup.html
dr.get(url);
dr.findElement(By.id("51")).click();
//得到当前窗口的句柄
String currentWindow = dr.getWindowHandle();
//得到所有窗口的句柄
Set<String> handles = dr.getWindowHandles();
Iterator<String> it = handles.iterator();  while(it.hasNext()){
String handle = it.next();  if(currentWindow.equals(handle)) continue;  WebDriver
window = dr.switchTo().window(handle);  System.out.println("title=
"+window.getTitle());
System.out.println ("url="+window.getCurrentUrl());


}
}
```

弹出窗口

- 捕获或者说定位弹出窗口的关键在于获得弹出窗口的句柄
- 在上面的代码里，使用windowhandle方法来获取当前浏览器窗口的句柄，使用了windowhandles方法获取所有弹出的浏览器窗口的句 柄，然后通过排除当前句柄的方法来得到新开窗口的句柄
- 在获取新弹出窗口的句柄后，使用switchto.window(newwindow_handle)方法，将新窗口的句柄作为参数传入既可捕获到新窗口了
- 如果想回到以前的窗口定位元素，那么再调用1次switch_to.window 方法，传入之前窗口的句柄既可达到目的。

处理alert、confirm、prompt对话框

- alert、confirm、prompt这样的js对话框在selenium1.X时代也是难啃的骨头，常常要用 autoit来帮助处理。selenium webdriver中处理这 些对话框十分方便简洁。以下面Dialogs.html代码 为例：

```
<html>
<head>
<title>Alert</title>
</head>

<body>
<input id = "alert" value = "alert" type = "button" onclick = "alert('欢迎！请按确认 继续！');"/>
<input id = "confirm" value = "confirm" type = "button" onclick = "confirm('确定 吗？');"/>
<input id = "prompt" value = "prompt" type = "button" onclick = "var name = prompt('请输入你的名字:','请输入你的名字'); document.write(name) "/>
</body>
</html>
```

html代码在页面上显示了三个按钮，点击他们分别弹出alert、confirm、prompt对话框。如果在prompt对话框中输入文字点击确定之后，将会刷新页面，显示出这些文字 。

测试代码

```
public class DialogsTest {
public static void main(String[] args) {  WebDriver dr = new
FirefoxDriver();
String url = "file://C:/Users/LRQ/workspace/2013-selenium-
webdriver-2/Dialogs.html";
dr.get(url);

//点击第一个按钮，输出对话框上面的文字，然后叉掉
dr.findElement(By.id("alert")).click();
Alert alert = dr.switchTo().alert();
String text = alert.getText();
System.out.println(text);
alert.dismiss();
```

测试代码

```
//点击第二个按钮，输出对话框上面的文字，然后点击确认
dr.findElement(By.id("confirm")).click();
Alert confirm = dr.switchTo().alert();
String text1 = confirm.getText();
System.out.println(text1);
confirm.accept();

//点击第三个按钮，输入你的名字，然后点击确认，最后
dr.findElement(By.id("prompt")).click();
Alert prompt = dr.switchTo().alert();
String text2 = prompt.getText();
System.out.println(text2);
prompt.sendKeys("jarvi");
prompt.accept();
}
}
```

处理alert、confirm、prompt对话框

- 从以上代码可以看出dr.switchTo().alert();这句可以得到alert\confirm\prompt对话框的对象，然后运用其方法对它进行操作。对话框操作的主要方法有：

getText() 得到它的文本值

accept() 相当于点击它的"确认"

dismiss() 相当于点击"取消"或者叉掉对话

sendKeys() 输入值

处理select下拉框

- 下面来看一下selenium webdriver是如何来处理select下拉框的，以 select1.html为例。

```
<html>
<head>
<style type="text/css">  div#container{width:500px}
div#header {background-color:#99bbbb;}
div#car-menu {background-color:#ffff99; height:200px; width:200px;
float:left;}  div#content {background-color:#EEEEEE; height:200px;
width:300px; float:left;}  div#footer {background-color:#99bbbb; clear:both;
text-align:center;}
h1 {margin-bottom:0;}
h2 {margin-bottom:0; font-size:14px;}  ul {margin:0;}
li {list-style:none;}
</style>
</head>
```

处理select下拉框

```
<body>

<div id="container">

<div id="header">
<h1>Main Title of Car Select</h1>
</div>

<div id="car-menu">
<h2>品牌选择</h2>
<select name="cars",id="select">
<option value="volvo">Volvo</option>
<option value="saab">Saab</option>
<option value="fiat" selected="selected">Fiat</option>
<option value="audi">Audi</option>
<option value="bmw">BMW</option>
<option value="Mercedes Benz ">Mercedes Benz </option>
</select>
</div>
```

处理select下拉框

```
<div id="content">
<h2>型号选择</h2>
<select name="engin",id="engin-select">
<option value="1"> 1.6T</option>
<option value="2">1.8T</option>
<option value="3" selected="selected">2.0</option>
<option value="4">2.5</option>
<option value="5">3.0</option>
<option value="6"> 3.5以上 </option>
</select>
</div>

<div id="footer">Copyright Software School SJTU
</div>

</div>

</body>
</html>
```

测试

```
public class SelectsTest {
public static void main(String[] args) {  WebDriver dr = new FirefoxDriver();
dr.get("file://E:/软件测试课程/selenium/select1.html");

// 通过下拉列表中选项的索引选中第二项，
Select selectCar = new Select(dr.findElement(By.name("cars")));
selectCar.selectByIndex(4);

// 通过可见文字“2.5”选中相应项，
Select selectengin = new Select(dr.findElement(By.name("engine")));
selectengin.selectByVisibleText("2.5");

}
}
```

操作Cookies

- Web 测试中我们经常会接触到Cookies，一个 Cookies 主要属性有”所在域、name、value、有效日期和路径"，下面来讲一下怎么操作Cookies。

测试

```
public class CookiesStudy {
    public static void main(String[] args) {
        WebDriver dr = new FirefoxDriver();
        dr.get("http://www.51.com");
        // 增加一个name = "name",value="value"的cookie
        Cookie cookie = new Cookie("name", "value");
        dr.manage().addCookie(cookie);
// 得到当前页面下所有的cookies，并且输出它们的所在域、name、value、有效日期和路径
        Set<Cookie> cookies = dr.manage().getCookies();
        System.out.println(String
                                .format("Domain -> name -> value -> expiry -> path"));
        for (Cookie c : cookies)
                    System.out.println(String.format("%s -> %s -> %s -> %s -> %s",
        c.getDomain(), c.getName(), c.getValue(), c.getExpiry(),c.getPath()));
        // 删除cookie有三种方法
        // 第一种通过cookie的name
        dr.manage().deleteCookieNamed("CookieName");
        // 第二种通过Cookie对象
        dr.manage().deleteCookie(cookie);
        // 第三种全部删除
         dr.manage().deleteAllCookies();
}
```

拖放元素

- selenium webdriver可以实现把一个元素拖放到另一个元素里面

拖放元素

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.interactions.Actions;

public class DragAndDrop {

    public static void main(String[] args){

        WebDriver dr = new FirefoxDriver();
        dr.get("http://koyoz.com/demo/html/drag-drop/drag-drop.html");

        //首先new出要拖入的页面元素对象和目标对象，然后进行拖入。
        WebElement  element = dr.findElement(By.id("item1"));
        WebElement  target = dr.findElement(By.id("drop"));
        (new Actions(dr)).dragAndDrop(element, target).perform();

        //利用循环把其它item也拖入
        String id="item" ;
        for(int i=2;i<=6;i++){
            String item = id+i;
```

dragAndDrop(element, target)这个方法是定义了“点击**element**元素对象，然后保持住，直到拖到目标元素对象里面才松开”这一系列动作的**Actions,** 如果你不调用**perform()**方法，这个**Actions**是不会执行的。

```
}
```



第三部分

1. 如何等待页面元素加载完成
2. 验证码
3. 利用selenium-webdriver截图
4. 如何处理table
5. 利用Actions类模拟鼠标和键盘的操作
6. 启用firefox代理
7. 启用默认情况下被firefox禁用的功能
8. 临时指定插件

如何等待页面元素加载完成

- web的自动化测试中，经常会遇到这样一种情况：当程序执行时需要页面某个元素，而此时这个 元素还未加载完成，这时程序就会报错。
- 等待元素出现后再进行对这个元素的操作。
- 在selenium-webdriver中我们用两种方式进行等待：
  - 明确的等待
  - 隐性的等待

明确的等待

- 明确的等待是指在代码进行下一步操作之前等待某一个条件的发生。
- 最不好的情况是使用Thread.sleep()去设置一段确认的时间去等待。
- selenium webdriver提供了一些方法帮助等待正好需要等待的时间。
  - WebDriverWait类
  - ExpectedCondition接口。

实例

- 下面的html代码实现了这样的一种效果：点击 click按钮4秒钟后，页面上会出现一个蓝色的div 块。
- 需要写一段自动化脚本去捕获这个出现的div，然后高亮它。

Wait.html

```
<html>
<head>
<title>Set Timeout</title>
<style>
.blue_box {background-color: blue; width = 50%; height: 150px; border: none;}
</style>
<script>
function show_div(){
setTimeout("create_div()", 4000);
}

function create_div(){
d = document.createElement('div');  d.className = “blue_box";
document.body.appendChild(d);
}
</script>
</head>
<body>
<button id = "b" onclick = "show_div()">click</button>
</body>
```

测试

```
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;  import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;  import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.ExpectedCondition;  import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitForSomthing1 {
public static void main(String[] args) throws Exception {

WebDriver dr = new FirefoxDriver();
String url = "file://E:/软件测试课程/selenium/Wait.html";
dr.get(url);
dr.findElement(By.id("b")).click();

WebDriverWait wait = new WebDriverWait(dr, 10);
wait.until(new ExpectedCondition<WebElement>() {
            public WebElement apply(WebDriver d) {
            return d.findElement(By.cssSelector(".blue_box"));
            }
            });
    WebElement element = dr.findElement(By.cssSelector(".red_box"));
    ((JavascriptExecutor) dr).executeScript("arguments[0].style.border = \"5px solid yellow\"", element);
} //main

}
```

代码WebDriverWait类的构造方法接受了一个WebDriver对象和一个等待最长时间（10秒）。然后调用until方法，其中重写了ExpectedCondition接口中的apply方法，让其返回一个WebElement，即加载完成的元素。默认情况下，WebDriverWait每500毫秒调用一次ExpectedCondition，直到有成功的返回，当然如果超过设定的值还没有成功的返回，将抛出异常。

隐性等待

- 隐性等待是指当要查找元素，而这个元素没有马上出现时，告诉WebDriver查询Dom一定时间。
- 默认值是0,但是设置之后，这个时间将在 WebDriver对象实例整个生命周期都起作用。上面的代码可改为如下代码：

测试

```
public class wait {
public static void main(String[] args){
System.out.println("sample5");

WebDriver dr = new FirefoxDriver();
// 设置10秒
dr.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);

String url = "file://E:/软件测试课程/selenium/Wait.html";  dr.get(url);
dr.findElement(By.id("b")).click();
// 注释掉原来的
/* WebDriverWait wait = new WebDriverWait (dr, 10);
wait.until(new ExpectedCondition<WebElement>() {  public
WebElement apply(WebDriver d) {
return d.findElement(By.cssSelector(".blue_box"));
}
});              */

WebElement element = dr.findElement(By.cssSelector(".blue_box"));
((JavascriptExecutor) dr).executeScript(
"arguments[0].style.border = \"5px solid yellow\"", element);


  }
}
```

验证码

- 等待手动输入验证码
- 使用pytesser识别验证码
- 调用云服务识别验证码

利用selenium-webdriver截图

- 在自动化测试中常常会用到截图功能。最近用了 一下selenium-webdriver的截图功能还算不错， 可以截取页面全图，不管页面有多长。

测试

```
public class ShotScreen {

public static void main(String[] args) throws IOException,
InterruptedException {  WebDriver dr = new FirefoxDriver();
dr.get("http://www.51.com");

// 这里等待页面加载完成
Thread.sleep(5000);

// 下面代码是得到截图并保存在D盘下
File screenShotFile = ((TakesScreenshot) dr).
getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(screenShotFile, new File("E:/软件测试课程
/selenium/test.png"));

}
}
```

如何处理table

- 前在selenium RC 里面有一个getTable方法，是 得到一个单元格中的文本 。
- 在selenium webdriver里，没有这样的方法，也就 是说没有专门操作table的类。
- 可以自己封装一个，以getTable方法为例，自己 也可以创建这样功能的一个方法。

table.html

```
<html>
<head>
<title>Table</title>

</head>
<body>
<table border="1" id="myTable">
<tr>
<th>Heading(row 0 ,cell 0)</th>
<th>Another Heading(row 0 ,cell 1)</th>
<th>Another Heading(row 0 ,cell 2)</th>
</tr>
<tr>
<td>row 1, cell 0</td>
<td>row 1, cell 1</td>
<td>row 1, cell 2</td>
</tr>
<tr>
<td>row 2, cell 0</td>
<td>row 2, cell 1</td>
<td>row 2, cell 2</td>
</tr>
</table>
</body>
</html>
```

测试

```
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;  import
org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class Table {  private WebDriver driver;

Table(WebDriver driver) {
this.driver = driver;
}
public String getCellText(By by, String tableCellAddress) {
// 得到table元素对象
WebElement table = driver.findElement(by);
// 对所要查找的单元格位置字符串进行分解，得到其对应行、列。
int index = tableCellAddress.trim().indexOf('.');
int row = Integer.parseInt(tableCellAddress.substring(0, index));
int cell = Integer.parseInt(tableCellAddress.substring(index + 1));
// 得到table表中所有行对象，并得到所要查询的行对象。
List<WebElement> rows = table.findElements(By.tagName("tr"));
WebElement theRow = rows.get(row);
// 调用getCell方法得到对应的列对象，然后得到要查询的文本。
String text = getCell(theRow, cell).getText();  return text;
}
```

测试

```
private WebElement getCell(WebElement Row, int
cell) {  List<WebElement> cells;
WebElement target = null;
// 列里面有"<th>"、"<td>"两种标签，所以分开处理。
if (Row.findElements(By.tagName("th")).size() > 0) {
cells = Row.findElements(By.tagName("th"));
target = cells.get(cell);
}
if (Row.findElements(By.tagName("td")).size() > 0) {
cells = Row.findElements(By.tagName("td"));
target = cells.get(cell);
}
return target;
}

public static void main(String[] args) {  WebDriver
driver;
driver = new FirefoxDriver();
driver.get("file://E:/软件测试课程
/selenium/table.html");

Table table = new Table(driver);  By by =
By.id("myTable");  String address = "1.1";

System.out.println(table.getCellText(by, address));
}

}
```

利用Actions类模拟鼠标和键盘的操作

- actions类，主要定义了一些模拟用户的鼠标 mouse，键盘keyboard操作。对于这些操作，使用perform()方法进行执行。
- actions类可以完成单一的操作，也可以完成几个操作的组合

单一的操作

- 单一的操作是指鼠标和键盘的一个操作。如鼠标左键按下、弹起或输 入一个字符串等。
- 涉及到鼠标键盘操作的一些方法，都可以使用actions类中的方法实现，比如：click，sendkeys。

```
WebElement        element = dr.findElement(By.id("test"));
WebElement        element1 = dr.findElement(By.id("test1"));
element.sendKeys("test");
element1.click;

//新建一个action
Actions action=new Actions(driver);
//操作
WebElement element=dr.findElement(By.id("test"));  WebElement
element1=dr.findElement(By.id("su"));  action.sendKeys(element,"test").perform();
action.moveToElement(element1);  action.click().perform();
```

组合操作

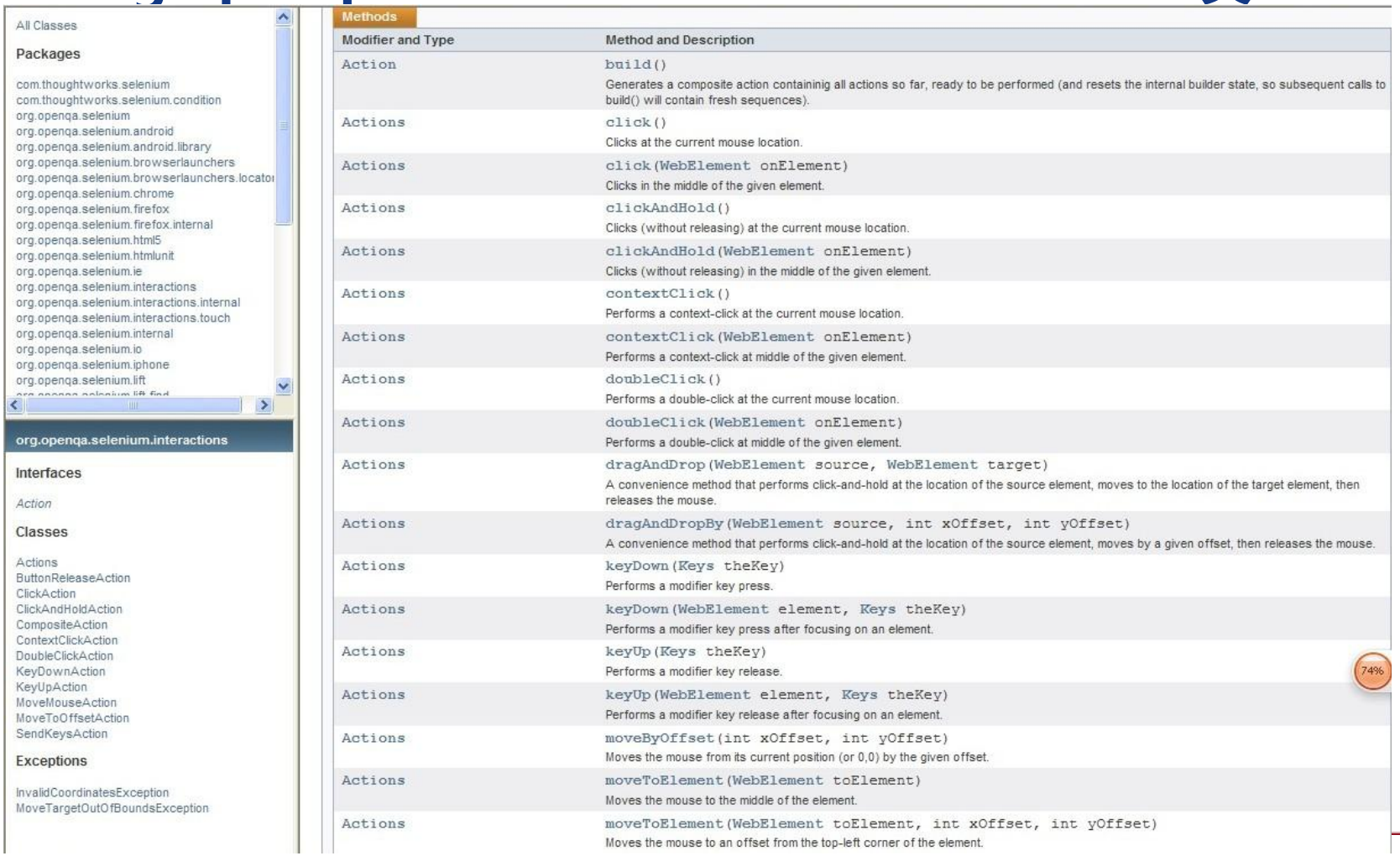
- 组合操作就是几个动作连在一起进行操作。如对 一个元素的拖放。

```
(new Actions(dr)).dragAndDrop(dr.findElement(By.id(item)),
target).perform();
```

```
Action dragAndDrop = builder.clickAndHold(someElement)
        .moveToElement(otherElement)
        .release(otherElement)
        .build().perform();
```

其他

- 其他鼠标或键盘操作方法可以具体看一下API里面的org.openqa.selenium.interactions.Actions类



All Classes

Packages

com.thoughtworks.selenium
com.thoughtworks.selenium.condition
org.openqa.selenium
org.openqa.selenium.android
org.openqa.selenium.android.library
org.openqa.selenium.browserlaunchers
org.openqa.selenium.browserlaunchers.locator
org.openqa.selenium.chrome
org.openqa.selenium.firefox
org.openqa.selenium.firefox.internal
org.openqa.selenium.html5
org.openqa.selenium.htmlunit
org.openqa.selenium.ie
org.openqa.selenium.interactions
org.openqa.selenium.interactions.internal
org.openqa.selenium.interactions.touch
org.openqa.selenium.internal
org.openqa.selenium.io
org.openqa.selenium.iphone
org.openqa.selenium.lift

org.openqa.selenium.interactions

Interfaces

Action

Classes

Actions
ButtonReleaseAction
ClickAction
ClickAndHoldAction
CompositeAction
ContextClickAction
DoubleClickAction
KeyDownAction
KeyUpAction
MoveMouseAction
MoveToOffsetAction
SendKeysAction

Exceptions

InvalidCoordinatesException
MoveTargetOutOfBoundsException

Methods

Modifier and Type	Method and Description
Action	build() Generates a composite action containinig all actions so far, ready to be performed (and resets the internal builder state, so subsequent calls to build() will contain fresh sequences).
Actions	click() Clicks at the current mouse location.
Actions	click(WebElement onElement) Clicks in the middle of the given element.
Actions	clickAndHold() Clicks (without releasing) at the current mouse location.
Actions	clickAndHold(WebElement onElement) Clicks (without releasing) in the middle of the given element.
Actions	contextClick() Performs a context-click at the current mouse location.
Actions	contextClick(WebElement onElement) Performs a context-click at middle of the given element.
Actions	doubleClick() Performs a double-click at the current mouse location.
Actions	doubleClick(WebElement onElement) Performs a double-click at middle of the given element.
Actions	dragAndDrop(WebElement source, WebElement target) A convenience method that performs click-and-hold at the location of the source element, moves to the location of the target element, then releases the mouse.
Actions	dragAndDropBy(WebElement source, int xOffset, int yOffset) A convenience method that performs click-and-hold at the location of the source element, moves by a given offset, then releases the mouse.
Actions	keyDown(Keys theKey) Performs a modifier key press.
Actions	keyDown(WebElement element, Keys theKey) Performs a modifier key press after focusing on an element.
Actions	keyUp(Keys theKey) Performs a modifier key release.
Actions	keyUp(WebElement element, Keys theKey) Performs a modifier key release after focusing on an element.
Actions	moveByOffset(int xOffset, int yOffset) Moves the mouse from its current position (or 0,0) by the given offset.
Actions	moveToElement(WebElement toElement) Moves the mouse to the middle of the element.
Actions	moveToElement(WebElement toElement, int xOffset, int yOffset) Moves the mouse to an offset from the top-left corner of the element.

74%

启用firefox代理

```
String PROXY = "localhost:8080";//如果不是本机，localhost替换成IP地址

org.openqa.selenium.Proxy proxy = new org.openqa.selenium.Proxy(); proxy.setHttpProxy(PROXY)
.setFtpProxy(PROXY)
.setSslProxy(PROXY);
DesiredCapabilities cap = new DesiredCapabailities();
cap.setPreference(CapabilityType.PROXY, proxy); WebDriver driver = new FirefoxDriver(cap);
```

启用默认情况下被firefox禁用的功能

```
FirefoxProfile profile = new
FirefoxProfile();
profile.setEnableNativeEvents(true);
WebDriver driver = new
FirefoxDriver(profile);
```

临时指定插件

- 有时需要临时让启动的firefox带一个插件，如 firebug，来定位问题等。首先要下载这个插件的xpi 安装包。剩下的就让selenium webdriver 来完成，如下：

```
FirefoxProfile firefoxProfile = new FirefoxProfile();
  firefoxProfile.addExtension(file);
  firefoxProfile.setPreference("extensions.firebug.currentVersion",
  "1.10.1"); //避免启动画面
WebDriver driver = new FirefoxDriver(firefoxProfile);
```